

Vampire Hunting v1.4.4 – Code Structure and Analytical Logic

Yixue Zhao

Introduction

The Vampire Hunting v1.4.4 project is a structured analytical system created to model how vampirism spreads within a defined population over several days. It reads text-based datasets containing participants' morning test results and evening contact groups, then applies logical reasoning and iterative deduction to infer infection timelines, identify transformation moments, and determine potential vampire sires—the originators of each infection chain.

Each stage of the program mirrors a distinct part of the Vampire Hunting assignment. From initial data validation to final cyclic analysis, every section builds upon the previous one, refining the logical framework until the infection pattern becomes consistent and complete. What follows is a detailed interpretation of each section's purpose and its analytical role in the overall system.

Section-by-Section Breakdown

Section 2: File Validation

Function: `file_exists(file_name)`

Before any computation begins, this section ensures the input file is accessible and correctly named. If the file cannot be located, the program halts execution. This safeguard prevents invalid or missing data from corrupting subsequent analyses.

The code I wrote:

```
# Section 2
```

```
def file_exists(file_name):
```

```
    """Verify that the file exists.
```

```
    Args:
```

```
        file_name (str): name of the file
```

```
    Returns:
```

```
        boolean: returns True if the file exists and False otherwise.
```

```
    """
```

```
    file_exists_result = os.path.isfile(file_name) # True if file exists, False otherwise
```

```
    return file_exists_result
```

Section 3: File Parsing

Functions: `parse_file()`, `read_participants()`, `read_num_days()`, `read_day_data()`,
`read_am_test_results()`, `read_pm_contact_groups()`

This section reads the input file and transforms raw text into structured data. It identifies all participants, determines how many days of information are provided, loads morning test results from `read_am_test_results()`, and constructs evening contact groups from `read_pm_contact_groups()`. The function `parse_file()` returns two key objects: the list of participants and the list of days, which together form the foundation for all later analysis.

Code I wrote:

#Section 3

import sys

def parse_file(file_name):

"""

input the file, parse the contents and return the new structure
 that contain the related datas for the vampire infiltration.

Args:

file_name (str): Contains the number of the names.

Returns:

participants (list): List of participant names.

days (list): Each element is a tuple:

(am_test_result, contact_groups)

am_test_result (dict): Matching of participant names to True/False

True if "V" --- indicate the Vampire, False if "H" --- indicate the Human

contact_groups (list): List of contact groups which shows the list of the name.

"""

try:

with open(file_name, 'r') as file:

participants = read_participants(file)

num_days = read_num_days(file)

days = [read_day_data(file, participants) for _ in range(num_days)]

return participants, days

except Exception as e:

print("Error found in file, aborting.")

raise

def read_participants(file):

"""

Read and parse the participants from the first line of the file.

Args:

file (file object): The file which will be read.

Returns:

list: List of participant names.

Raises:

ValueError: If the participants line is missing or empty. the ValueError will show.

.....

```
participants_line = file.readline().strip()
```

```
if not participants_line:
```

```
    raise ValueError("No participants line found.")
```

```
participants = [name.strip() for name in participants_line.split(",")]
```

```
if not participants:
```

```
    raise ValueError("Participants list is empty.")
```

```
return participants
```

```
def read_num_days(file):
```

```
    .....
```

Read and parse the number of days from the second line of the file.

Args:

file (file object): The file which will be read.

Returns:

int: input of number of days.

Raises:

ValueError: If the number of days is missing or not a valid integer, the ValueError will show.

```
.....
```

```
num_days_line = file.readline().strip()
```

```
if not num_days_line.isdigit():
```

```
    raise ValueError("Number of days is not a valid integer.")
```

```
return int(num_days_line)
```

```
def read_day_data(file, participants):
```

```
    .....
```

Read and parse data for a single day, including AM test results and contact groups.

Args:

file (file object): The file being read.

participants (list): List of valid participant names.

Returns:

tuple: (am_test_result, contact_groups)

am_test_result (dict): Mapping of participant names to True/False which show the status.

contact_groups (list): List of contact groups, each a list of participant names.

Raises:

ValueError: If any part of the day's data is invalid.

.....

```
am_test_result = read_am_test_results(file, participants)
```

```
pm_contact_groups = read_pm_contact_groups(file, participants)
```

```
return am_test_result, pm_contact_groups
```

```
def read_am_test_results(file, participants):
```

```
.....
```

Read and parse the AM test results line.

Args:

file (file object): The file being read.

participants (list): List of valid participant names.

Returns:

dict: Mapping of participant names to True/False

True if vampire--V, False if human---H.

Raises:

ValueError: If any test entry is invalid or participant unknown.

.....

```
am_test_line = file.readline().strip()
```

```
if am_test_line == "###":
```

```
    # No test results for the day
```

```
    return {}
```

```
am_test_result = {}
```

```
entries = [entry.strip() for entry in am_test_line.split(",") if entry.strip()]
```

```
for entry in entries:
```

```
    if ':' not in entry:
```

```
        raise ValueError(f"Invalid test entry format: '{entry}'. Expected 'Name:Result'.")
```

```
    name, result = [part.strip() for part in entry.split(':', 1)]
```

```
    if name not in participants:
```

```

        raise ValueError(f"Unknown participant: '{name}'.")

    if result not in {'H', 'V'}:
        raise ValueError(f"Invalid test result '{result}' for participant '{name}'.")

    # Map 'V' to True and 'H' to False
    am_test_result[name] = (result == 'V')

return am_test_result

def read_pm_contact_groups(file, participants):
    """
    Read and parse the contact groups for each day.

    Args:
        file (file object): The file is input and read.
        participants (list): List of present participant names.

    Returns:
        list: List of contact groups which shows that a list of participant names.

    Raises:
        ValueError: If the number of contact groups is invalid or if any group line is empty
        or contains unknown participants.
    """
    contact_group_nums_line = file.readline().strip()
    if not contact_group_nums_line.isdigit():
        raise ValueError("Invalid number of contact groups.")

    contact_group_nums = int(contact_group_nums_line)
    contact_groups = []

    for i in range(contact_group_nums):
        group_line = file.readline().strip()
        if not group_line:
            raise ValueError(f"Contact group {i + 1} is empty.")

        group = [name.strip() for name in group_line.split(",") if name.strip()]

        if not group:
            raise ValueError(f"Contact group {i + 1} contains no valid participants.")

        for name in group:

```

```

        if name not in participants:
            raise ValueError(f"Unknown participant '{name}' in contact group {i + 1}.")

    contact_groups.append(group)

return contact_groups

```

Section 4: Data Display

Functions: `print_header()`, `print_participants_info()`, `print_day_overview()`, `print_tests()`, `print_contact_groups()`, `pretty_print_infiltration_data()`

Here, the program prints the parsed data in a clear, human-readable way. It displays participant names, day counts, AM/PM information, and group structures. This visual confirmation step allows users to verify the dataset before any reasoning or propagation takes place.

Code I wrote:

Section 4

def print_header():

"""

Print the header line.

"""

print("Vampire Infiltration Data")

def print_participants_info(participants, days):

"""

Print the total number of days and the list of participants.

"""

num_days = len(days)

sorted_participants = sorted(participants)

day_word = "day" if num_days == 1 else "days"

print(f"{num_days} {day_word} with the following participants:
{format_list(sorted_participants)}.")

def print_day_overview(day_num, tests, contact_groups):

"""

Print an overview line for a single day including the number of tests and contact groups.

"""

num_tests = len(tests)

num_groups = len(contact_groups)

tests_word = "vampire test" if num_tests == 1 else "vampire tests"

groups_word = "contact group" if num_groups == 1 else "contact groups"

print(f"Day {day_num} has {num_tests} {tests_word} and {num_groups} {groups_word}.")

```

def print_tests(tests):
    """
    Print the vampire test results fpr each day.
    """
    num_tests = len(tests)
    tests_plural = "test" if num_tests == 1 else "tests"
    print(f" {num_tests} {tests_plural}")
    if num_tests > 0:
        sorted_test_participants = sorted(tests.keys())
        for name in sorted_test_participants:
            status = "a vampire!" if tests[name] else "human."
            print(f" {name} is {status}")

```

```

def print_contact_groups(contact_groups):
    """
    Print the contact groups for a single day.
    """
    num_groups = len(contact_groups)
    groups_plural = "group" if num_groups == 1 else "groups"
    print(f" {num_groups} {groups_plural}")
    for group in contact_groups:
        if len(group) == 1:
            group_str = group[0]
        else:
            group_str = format_list(group)
        print(f" {group_str}")

```

```

def pretty_print_infiltration_data(data):

    participants, days = data

    print_header()
    print_participants_info(participants, days)

    for day_num, (tests, contact_groups) in enumerate(days, start=1):
        print_day_overview(day_num, tests, contact_groups)
        print_tests(tests)
        print_contact_groups(contact_groups)

    print("End of Days")

```

Section 5: Time-Based Contact Lookup

Function: `contacts_by_time()`

This section defines the procedure that identifies which contact group a participant belongs to at a given time step `t`.

When the time corresponds to the initial state (`is_initial(t)` is true), the function returns an empty list, since no contacts exist before the first day begins.

For any other time, `day_of_time(t)` is used to determine the corresponding day index, and the function accesses the contact groups recorded for that day.

If the index lies outside the range of available data, the function again returns an empty list. It then searches through all contact groups of the selected day and returns the one that contains the specified participant.

If the participant is not found in any group, an empty list is returned.

The logic relies only on `is_initial(t)` and `day_of_time(t)` to manage the mapping between time units and daily contact structures.

Section 5

```
def contacts_by_time(participant, t, contact_groups):
```

```
    """
```

```
        Read the contact group for a specific participant at a given time.
```

```
    Args:
```

```
        participant (Any): The individual whose in the contact group will be found.
```

```
        t (Any): The input of time used to determine the a day.
```

```
        contact_groups (List[List[List[Any]]]):
```

```
            A nested list where each sublist represents a day, and each day contains groups of participants.
```

```
    Returns:
```

```
        List[Any]: The group containing the participant for the day, or an empty list if not found.
```

```
    """
```

```
    if is_initial(t):
```

```
        return []
```

```
    d = day_of_time(t)
```

```
    d_index = d - 1
```

```
    if d_index < 0 or d_index >= len(contact_groups):
```

```
        return []
```

```
    groups_for_day = contact_groups[d_index]
```

```
    for grp in groups_for_day:
```

```

        if participant in grp:
            return grp

    return []

```

Section 6: Vampire Knowledge Initialization

Functions: `create_initial_vk()`, `pretty_print_vampire_knowledge()`, `pretty_print_vks()`

This stage initializes the first version of the vampire knowledge tables (vks). Each vk entry records the known state of every participant: 'H' for human, 'V' for vampire, or 'U' for unknown. Initially, everyone is marked as unknown. The pretty-print functions display this information in a tabular form for inspection.

Code I wrote:

Section 6

```
def create_initial_vk(participants):
```

```
    """
```

```
    A blank vampire status table was created for the participants who will be tested.
```

```
    """
```

```
    vk = {participant: "U" for participant in participants}
```

```
    return vk
```

```
def pretty_print_vampire_knowledge(vk):
```

```
    """ Pretty-print the vampire status table (vk).
```

```
    """
```

```
    humans = sorted([name for name in vk if vk[name] == "H"])
```

```
    vampires = sorted([name for name in vk if vk[name] == "V"])
```

```
    unknowns = sorted([name for name in vk if vk[name] == "U"])
```

```
    print(f"   Human{'s' if len(humans) != 1 else ''}: {format_list(humans)}")
```

```
    print(f"   Unclear individual{'s' if len(unknowns) != 1 else ''}: {format_list(unknowns)}")
```

```
    print(f"   Vampire{'s' if len(vampires) != 1 else ''}: {format_list(vampires)}")
```

Done by professors

```
def pretty_print_vks(vks):
```

```
    print(f'Vampire Knowledge Tables')
```

```
    for i in range(len(vks)):
```

```
        print(f'Day {str_time(i)}:')
```

```
        pretty_print_vampire_knowledge(vks[i])
```

```
    print(f'End Vampire Knowledge Tables')
```

Section 7: Applying Test Results

Functions: update_vk_with_tests(), validate_participants(), determine_new_status(),
apply_tests()

Morning (AM) test data are applied here to update the vampire knowledge tables (vks). The program ensures internal consistency—no participant can revert from vampire to human, and contradictions between test data and prior states trigger an error. This section solidifies each day's verified results in the timeline.

Code I wrote:

Section 7

```
def validate_participants(vk, tests):
```

```
    """
```

```
    Test that all tested names are present in the vk dictionary.
```

```
    If participant is not in the vk, print error and exit the system.
```

```
    """
```

```
    for name in tests:
```

```
        if name not in vk:
```

```
            print("Error found in data: test subject is not a participant; aborting.")
```

```
            sys.exit()
```

```
def determine_new_status(curr_status, result):
```

```
    """
```

```
    Given the current status of a participant and their test result,
```

```
    to determine the new status.
```

```
    If contradictions occur, print error and exit.
```

```
    """
```

```
    if curr_status == "U":
```

```
        if result:
```

```
            return "V"
```

```
        else:
```

```
            return "H"
```

```
    if curr_status == "H":
```

```
        if result:
```

```
            print("Error found in data: humans cannot be vampires; aborting.")
```

```
            sys.exit()
```

```
        return "H"
```

```

    if curr_status == "V":
        if not result:
            print("Error found in data: vampires cannot be humans; aborting.")
            sys.exit()
        return "V"

    return curr_status

def apply_tests(vk, tests):
    """
    Apply the test results to the vampire status table and return the updated version.
    """
    updated_vk = vk.copy()
    for name, result in tests.items():
        curr_status = vk[name]
        updated_vk[name] = determine_new_status(curr_status, result)
    return updated_vk

def update_vk_with_tests(vk, tests):
    """
    Update the vampire status table with the given test results.
    """
    validate_participants(vk, tests)
    return apply_tests(vk, tests)

```

Section 8: Forward Propagation of Vampire Status

Function: `update_vk_with_vampires_forward()`

If a participant is already marked as a vampire at time t , this function ensures they remain a vampire at all future times. The forward propagation step maintains logical continuity so that once vampirism appears, it persists across subsequent entries in `vks`.

Code I wrote:

Section 8

```
def update_vk_with_vampires_forward(vk_pre, vk_post):
```

```
    """
```

```
    Update the vampire status table.
```

```
    Parameters
```

```
    -----
```

```
    vk_pre : dict
```

```
        The table list of vampire status of the previous time.
```

```

vk_post : dict
    The updated vampire status table after testing.
Returns
-----
dict
    The updated vampire status table (current time unit: t).
"""
updated_vk = vk_post.copy()
for name, pre_status in vk_pre.items():
    if pre_status == "V":
        post_status = vk_post.get(name)
        if post_status == "U":
            updated_vk[name] = "V"
        elif post_status == "H":
            print("Error found in data: vampires cannot be humans; aborting.")
            sys.exit()
return updated_vk

```

Section 9: Backward Propagation of Human Status

Function: `update_vk_with_humans_backward()`

Backward propagation guarantees that a participant confirmed human on a later day must also have been human before. This logic enforces temporal consistency for human states, filling in earlier entries of vks when later evidence confirms a participant's humanity.

Code I wrote:

Section 9

```

def update_vk_with_humans_backward(vk_pre, vk_post):
    """

```

Backward propagate the human status in the vampire status table.

If an individual is human later (in `vk_post`), they must also have been human earlier (in `vk_pre`).

If `vk_pre` was unknown (U), change it to human (H).

If `vk_pre` was vampire (V), produce an error.

Otherwise, do not change `vk_pre`.

```

    """

```

```

    updated_pre_vk = vk_pre.copy()

```

```

    for name, post_status in vk_post.items():

```

```

        if post_status == "H":

```

```

            pre_status = updated_pre_vk[name]

```

```

            if pre_status == "U":

```

```

                updated_pre_vk[name] = "H"

```

```

        elif pre_status == "V":
            print("Error found in data: humans cannot be vampires; aborting.")
            sys.exit()

    return updated_pre_vk

```

Section 10: Overnight Safety Propagation

Functions: `update_vk_overnight()`, `check_transition()`, `propagate_status()`

Overnight periods are treated as safe intervals with no new infections. This section ensures that participants retain their current status overnight—humans stay human, vampires remain vampires. The `check_transition()` and `propagate_status()` helpers handle validation and data transfer between consecutive time steps.

Code I wrote:

Section 10

```

def check_transition(status_pre, status_post):
    """
    Check for unable overnight transitions:
    - Human cannot become Vampire
    - Vampire cannot become Human
    """
    if status_pre == "H" and status_post == "V":
        print("Error found in data: humans cannot be vampires; aborting.")
        sys.exit()
    if status_pre == "V" and status_post == "H":
        print("Error found in data: vampires cannot be humans; aborting.")
        sys.exit()

```

```

def propagate_status(status_pre, updated_vk, person):
    """
    Propagate the status from the previous evening:
    - If pre was Human---H, remain Human---H next morning.
    - If pre was Vampire---V, remain Vampire---V next morning.
    - If pre was Unknown---U, leave status as is---do nothing.
    """
    if status_pre == "H":
        updated_vk[person] = "H"
    elif status_pre == "V":
        updated_vk[person] = "V"

```

```

def update_vk_overnight(vk_pre, vk_post):

```

.....

Update the vk structure overnight.

Everyone is safe at night.

Anyone who was human/vampire at the end of the day remains human/vampire the next morning.

Error conditions:

- If human at PM becomes vampire at AM, shows error
- If vampire at PM becomes human at AM, shows error

Otherwise, propagate:

- H stays H
- V stays V
- U remains whatever vk_post says, it will not change as there isn't driving force.

.....

```
updated_vk = vk_post.copy()
```

```
for person in vk_pre:
```

```
    status_pre = vk_pre[person]
```

```
    status_post = vk_post[person]
```

```
    # Check for invalid transitions
```

```
    check_transition(status_pre, status_post)
```

```
    propagate_status(status_pre, updated_vk, person)
```

```
return updated_vk
```

Section 11: Contact Group Updates

Functions: `update_vk_with_contact_group()`, `check_invalid_participants()`,
`vampire_forward_check()`, `handle_non_contact_participants()`, `process_contact_groups()`

This section incorporates PM contact group data into the reasoning process. If all members of a group are human, none can become vampires immediately afterward. The function `update_vk_with_contact_group()` checks group logic, manages participants with no recorded contacts, and applies forward consistency rules to refine each day's knowledge table.

Code I wrote:

```
# Section 11
```

```
def check_invalid_participants(vk_pre, contacts):
```

```
    .....
```

```
    Step 1: Check if participants exist in contact groups.
```

```

"""

participants = set(vk_pre.keys())
for group in contacts:
    for person in group:
        if person not in participants:
            print("Error found in data: contact subject is not a participant; aborting.")
            sys.exit()

def vampire_forward_check(updated_vk, vk_pre):
    """
    Step 2: Check the vampire-forward.
    If someone was V at pre and is H at post, show the error.
    If V at pre and U at post, show that set V at post.
    """
    for person in vk_pre:
        pre_status = vk_pre[person]
        post_status = updated_vk[person]

        if pre_status == "V" and post_status == "H":
            print("Error found in data: vampires cannot be humans; aborting.")
            sys.exit()

        if pre_status == "V" and post_status == "U":
            updated_vk[person] = "V"

def handle_non_contact_participants(updated_vk, vk_pre, contacts):
    """
    Step 3: For participants not in any contact group:
    - If H at pre and not in contacts, must remain or become H at post.
    - If V at pre and not in contacts and U at post, already handled above.
    - If U at pre and not in contacts, do nothing special.
    """
    all_in_contacts = {p for group in contacts for p in group}
    for person in vk_pre:
        if person not in all_in_contacts:
            pre_status = vk_pre[person]
            post_status = updated_vk[person]

            if pre_status == "H":
                if post_status == "V":
                    print("Error found in data: humans cannot be vampires; aborting.")
                    sys.exit()

```

```

        if post_status == "U":
            updated_vk[person] = "H"
        # Other cases require no additional action here.

```

```

def process_contact_groups(updated_vk, vk_pre, contacts):
    """
    Step 4: For each contact group:
    - If all H at pre, then all must be H at post, else error if any V at post.
    """
    for group in contacts:
        all_human_pre = all(vk_pre[p] == "H" for p in group)
        if all_human_pre:
            for p in group:
                if updated_vk[p] == "V":
                    print("Error found in data: humans cannot be vampires; aborting.")
                    sys.exit()
            for p in group:
                updated_vk[p] = "H"
        # If not all human at pre, do nothing special.

```

```

def update_vk_with_contact_group(vk_pre, contacts, vk_post):
    """
    Update vk after PM contacts.
    The logic and output remain the same, but now the code is broken down into steps.
    """
    updated_vk = vk_post.copy()

    check_invalid_participants(vk_pre, contacts)

    vampire_forward_check(updated_vk, vk_pre)

    handle_non_contact_participants(updated_vk, vk_pre, contacts)

    process_contact_groups(updated_vk, vk_pre, contacts)

    return updated_vk

```

Section 12: Infection Window Detection

Functions: find_infection_windows(), get_participants(), analyze_participant_timeline(), pretty_print_infection_windows()

The infection window (iw) represents when each participant could have transitioned from human to vampire. This section scans the full set of vks to determine the last confirmed human moment and the first confirmed vampire moment for every individual. The resulting windows provide the temporal framework for analyzing infections.

Code I wrote:

Section 12

```
def get_participants(vks):
```

```
    """Return participant name lists"""
```

```
    return list(vks[0].keys())
```

```
def analyze_participant_timeline(vks, name):
```

```
    """
```

```
    Analysis the timeline for a single participant.
```

```
    Returns:
```

```
    - first_human_time: The last time they were observed human---None if never human
```

```
    - first_vampire_time: The first time they were observed as vampire---None if never vampire
```

```
    """
```

```
    num_time_units = len(vks)
```

```
    first_human_time = None
```

```
    first_vampire_time = None
```

```
    for t in range(num_time_units):
```

```
        status = vks[t][name]
```

```
        if status == "H":
```

```
            # Keep updating so that at the end this is the *last* observed human time
```

```
            first_human_time = t
```

```
        elif status == "V":
```

```
            # Record the first vampire time and break
```

```
            first_vampire_time = t
```

```
            break
```

```
    return first_human_time, first_vampire_time
```

```
def find_infection_windows(vks):
```

```
    """
```

```
    Shows the potential infection windows for vampires,
```

```
    and using helper function to give a more clear and detail result.
```

```
    """
```

```
    windows = {}
```

```
    participants = get_participants(vks)
```

```

for name in participants:
    first_human_time, first_vampire_time = analyze_participant_timeline(vks, name)

    if first_vampire_time is not None:
        # If never observed as human, start from time 0
        if first_human_time is None:
            first_human_time = 0

    windows[name] = (first_human_time, first_vampire_time)

return windows

def pretty_print_infection_windows(iw):
    """Pretty-print the infection windows for the vampires."""
    for name in sorted(iw.keys()):
        start, end = iw[name]
        print(f" {name} was turned between day {str_time(start)} and day {str_time(end)}.")

```

Section 13: Potential Vampire Sires

Functions: `find_potential_sires()`, `get_contact_groups_for_time()`, `record_sire_information()`, `pretty_print_potential_sires()`

Given each participant's infection window (`iw`), this section identifies possible infectors, or "sires." Using PM contact data, the function `find_potential_sires()` compiles a dictionary (`ps`) showing which vampires were in contact with each participant during their potential infection periods. It sets the stage for tracing infection origins.

Code I wrote:

Section 13

```
def find_potential_sires(iw, groups):
```

```
    """
```

To find out the potential sires for each vampire during their infection window period.

Parameters

```
    -----
```

`iw` : dict

The infection windows for the vampires.

The key is a vampire's name, and the value is a tuple.

`groups` : list

List of contact groups for each day.

Each element in `groups` corresponds to a day,

and it shows a list of contact groups which contain the participant names.

Returns

dict

The potential sires for each vampire.

The key is a vampire's name, and the value is a list of tuples---time, group.

If the vampire was not part of any contact group during a PM period in their infection window,

append (time, []) to indicate no contacts.

.....

```
sires = {}
```

```
for name, (start, end) in iw.items():
```

```
    sires[name] = []
```

```
    for t in range(start, end + 1):
```

```
        if not period_of_time(t): # PM time unit
```

```
            contact_groups = get_contact_groups_for_time(t, groups)
```

```
            sire_entry = record_sire_information(name, t, contact_groups)
```

```
            sires[name].append(sire_entry)
```

```
return sires
```

```
def get_contact_groups_for_time(t, groups):
```

.....

Call back the contact groups for a specific time unit.

Parameters

t : int

The time unit for which to recall back contact groups.

groups : list

The list of contact groups for each day.

Returns

list

The contact groups active at time t.

Returns an empty list if no groups are found.

.....

```
day = day_of_time(t)
```

```
day_index = day - 1
```

```
if 0 <= day_index < len(groups):
```

```
    return groups[day_index]
```

```
return []
```

```

def is_vampire_in_any_group(name, contact_groups):
    """
    Check if a vampire is part of any contact group.
    """
    for group in contact_groups:
        if name in group:
            return True
    return False


def record_sire_information(name, t, contact_groups):
    """
    Record potential sires by listing other vampires in the same contact group.
    """
    for group in contact_groups:
        if name in group:
            return (t, group)
    return (t, [])


def pretty_print_potential_sires(ps):
    """
    Pretty print the potential sires for each vampire.
    """
    for name in sorted(ps.keys()):
        print(f" {name}:")
        if not ps[name]:
            print(f"    (None)")
        else:
            for time, contacts in ps[name]:
                day_str = str_time(time)
                if contacts:
                    contacts_str = format_list(sorted(contacts))
                    print(f"    On day {day_str}, met with {contacts_str}.")
                else:
                    print(f"    On day {day_str}, met with (None).")

```

Section 14: Sire Data Trimming

Functions: trim_potential_sires(), remove_vampire_self(), remove_definite_humans(), process_contacts()

The trimming process refines the ps data by removing impossible cases—such as a participant infecting themselves or confirmed humans listed as sires. This step ensures that only feasible infection links remain, tightening the logical connections between vampires and their possible sources.

Code I wrote:

Section 14

```
def remove_vampire_self(vamp, contacts):
```

```
    """
```

```
    Remove the vampire themselves from the contacts list.
```

```
    """
```

```
    return [contact for contact in contacts if contact != vamp]
```

```
def remove_definite_humans(contacts, vk):
```

```
    """
```

```
    The definite human will be remove from the contacts based on the vk status.
```

```
    """
```

```
    return [contact for contact in contacts if vk.get(contact, "Unknown") != "H"]
```

```
def is_valid_time(time, vks_length):
```

```
    """
```

```
    To check if the given time index is valid within the vks list.
```

```
    """
```

```
    return 0 <= time < vks_length
```

```
def process_contacts(vamp, contacts, vk):
```

```
    """
```

```
    Process the contacts by removing the vampire themselves and definite humans.
```

```
    """
```

```
    contacts_without_self = remove_vampire_self(vamp, contacts)
```

```
    final_contacts = remove_definite_humans(contacts_without_self, vk)
```

```
    return final_contacts
```

```
def trim_potential_sires(ps, vks):
```

```
    """
```

```
    Refactor the potential sires by removing empty days, the vampire themselves,
    and definite humans from the sire lists.
```

```
    """
```

```
    trimmed_ps = {}
```

```
    for vamp, time_contacts in ps.items():
```

```
        trimmed_contacts = []
```

```

    for time, contacts in time_contacts:
        vk = vks[time]
        processed_contacts = process_contacts(vamp, contacts, vk)

        if processed_contacts:
            trimmed_contacts.append((time, processed_contacts))

    trimmed_ps[vamp] = trimmed_contacts

return trimmed_ps

```

Section 15: Infection Window Refinement

Functions: trim_infection_windows(), has_sire_info(), get_sire_times(), adjust_start(), adjust_end()

Once sire information is available, infection windows (iw) are refined. The program adjusts start and end times to align with when potential sires were active. This synchronization improves temporal precision and helps isolate the most likely infection intervals.

Code I wrote:

Section 15

```
def has_sire_info(vamp, ps):
```

```
    """
```

```
    Check if a vampire has sire information.
```

```
    """
```

```
    return vamp in ps and len(ps[vamp]) > 0
```

```
def get_sire_times(vamp, ps):
```

```
    """
```

```
    Retrieve all sire times for a given vampire.
```

```
    """
```

```
    return [t for (t, c) in ps[vamp]]
```

```
def adjust_start(start, earliest_time):
```

```
    """
```

```
    To align the start of the infection window based on sire times.
```

```
    """
```

```
    if start > earliest_time:
```

```
        return earliest_time
```

```
    else:
```

```
        if period_of_time(start) and not is_initial(start):
```

```
            return start + 1
```

```
        else:
```

```

        return start

def adjust_end(end, latest_time):
    """
    To adjust the end of the infection window based on sire times.
    """
    if end > latest_time:
        return latest_time
    else:
        if period_of_time(end):
            return end
        else:
            return end

def trim_infection_windows(iw, ps):
    """
    Tighten infection windows based on the trimmed potential sires.

    If a vampire has no sire information, set window to (0,0).
    If there is sire information, find the earliest and latest PM time units from ps for that
    vampire.
    Set the infection window to that fixed range.

    """
    new_iw = {}
    for vamp, (start, end) in iw.items():
        if not has_sire_info(vamp, ps):
            new_iw[vamp] = (0, 0)
        else:
            # Retrieve all sire times
            times = get_sire_times(vamp, ps)
            earliest_time = min(times)
            latest_time = max(times)

            # Adjust the start and end of the infection window
            new_start = adjust_start(start, earliest_time)
            new_end = adjust_end(end, latest_time)

            new_iw[vamp] = (new_start, new_end)
    return new_iw

```

Section 16: Updating Knowledge Tables with Infection Windows

Functions: update_vks_with_windows(), enforce_human_before_start(),
enforce_vampire_after_end(), enforce_vampire_at_exact_time()

Here, infection-window information is integrated back into the main vampire knowledge tables (vks). Participants are forced to remain human before their infection window begins and vampire after it ends. If the start and end are identical, the infection is assigned to that exact moment. This ensures internal logical harmony across time steps.

Code I wrote:

Section 16

```
def update_vks_with_windows(vks, iw):
```

```
    """
```

```
    Update the vampire knowledge (vk) structures with the deduced infection windows.
```

```
    For each vampire v with window (start, end):
```

- All times < start: v must be human---H
- All times > end: v must be vampire---V
- If start == end: at that exact time unit v must be vampire---V

```
    If a contradiction occurs which human becomes vampire or vampire becomes human,  
    print an error message and exit the program.
```

```
    Counts the number of changes made.
```

```
    """
```

```
    changes = 0
```

```
    participants = vks[0].keys()
```

```
    num_times = len(vks)
```

```
    for vamp, (start, end) in iw.items():
```

```
        if vamp not in participants:
```

```
            continue
```

```
        changes += enforce_human_before_start(vamp, start, vks, num_times)
```

```
        changes += enforce_vampire_after_end(vamp, end, vks, num_times)
```

```
        changes += enforce_vampire_at_exact_time(vamp, start, end, vks)
```

```
    return (vks, changes)
```

```
def enforce_human_before_start(vamp, start, vks, num_times):
```

```
    """
```

Ensure that the vampire was human before the infection window starts.

Parameters

vamp : str

 Name of the vampire.

start : int

 Start time of the infection window.

vks : list of dict

 Vampire status structures.

num_times : int

 Total number of time units.

Returns

int

 Number of changes made.

.....

changes = 0

for t in range(0, start):

 status = vks[t][vamp]

 if status == "V":

 print("Error found in data: humans cannot be vampires; aborting.")

 sys.exit()

 elif status == "U":

 vks[t][vamp] = "H"

 changes += 1

return changes

def enforce_vampire_after_end(vamp, end, vks, num_times):

.....

Ensure that the vampire remains a vampire after the infection window ends.

.....

changes = 0

adjusted_end = adjust_end_time(end)

for t in range(adjusted_end + 1, num_times):

 status = vks[t][vamp]

 if status == "H":

 print("Error found in data: vampires cannot be humans; aborting.")

 sys.exit()

 elif status == "U":

```

        vks[t][vamp] = "V"
        changes += 1
    return changes

def adjust_end_time(end):
    """
    Adjust the end time if it's a PM time unit.
    """
    if not period_of_time(end):
        return end - 1
    return end

def enforce_vampire_at_exact_time(vamp, start, end, vks):
    """
    Ensure that the vampire is marked as a vampire at the exact time of infection.
    """
    changes = 0
    if start == end:
        status = vks[end][vamp]
        if status == "H":
            print("Error found in data: vampires cannot be humans; aborting.")
            sys.exit()
        elif status == "U":
            vks[end][vamp] = "V"
            changes += 1
    return changes

```

Section 17: Cyclic Analysis (Phase 1)

Function: `cyclic_analysis()`

The first stage of iterative reasoning begins here. The program repeatedly trims sire lists, refines infection windows, and updates vks until no further logical changes occur. This loop achieves a self-consistent state among all tables and datasets.

Code:

```

# Section 17; done by professors
def cyclic_analysis(vks, iw, ps):
    count = 0
    changes = 1
    while (changes != 0):
        ps = trim_potential_sires(ps, vks)

```

```

iw = trim_infection_windows(iw, ps)
(vks, changes) = update_vks_with_windows(vks, iw)
count = count + 1
return (vks, iw, ps, count)

```

Section 18: Vampire Strata Classification

Functions: `vampire_strata()`, `pretty_print_vampire_strata()`

Participants are divided into three strata based on their infection windows (iw): originals (infected before the dataset starts), newborns (infected within recorded time), and vamps_unclear (whose origin cannot be precisely determined). The pretty-print function outputs these classifications in readable form.

Code I wrote:

```
# Section 18: vampire strata
```

```
def vampire_strata(iw):
```

```
    """
```

```
    Determine vampire strata:
```

```
    - If (0,0): original
```

```
    - If start > 0: newborn
```

```
    - Otherwise: unknown strata
```

```
    """
```

```
    originals = set()
```

```
    vamps_unclear = set()
```

```
    newborns = set()
```

```
    for vamp, (start, end) in iw.items():
```

```
        if start == 0 and end == 0:
```

```
            originals.add(vamp)
```

```
        elif start > 0:
```

```
            newborns.add(vamp)
```

```
        else:
```

```
            vamps_unclear.add(vamp)
```

```
    return originals, vamps_unclear, newborns
```

```
def pretty_print_vampire_strata(originals, vamps_unclear, newborns):
```

```
    """
```

```
    Pretty-print vampire strata.
```

```
    Original vampires: ...
```

```
    Unknown strata vampires: ...
```

Newborn vampires: ...

Indent each line by two spaces. Sort names alphabetically.

```
"""
originals = sorted(originals)
unclear_vamps = sorted(vamps_unclear)
newborns = sorted(newborns)

print("  Original vampires: " + format_list(originals))
print("  Unknown strata vampires: " + format_list(unclear_vamps))
print("  Newborn vampires: " + format_list(newborns))
```

Section 19: Sire Set Calculation

Functions: `calculate_sire_sets()`, `pretty_print_sire_sets()`

This section consolidates the refined sire data into sire sets (ss). Each vampire is assigned a final set of plausible infectors derived from the previous trimming and analysis. It provides a concise summary of who may have infected whom, highlighting key relational links in the network.

Code I wrote:

```
# Section 19: vampire sire sets
def calculate_sire_sets(ps):
    """
    Calculate sire sets from ps.

    For each vampire v:
        Combine all contacts from ps[v] into the set
    Return a dict: {vamp: set_of_possible_sires}
    """
    ss = {}
    for vamp, entries in ps.items():
        sire_set = set()
        for (t, contacts) in entries:
            for c in contacts:
                sire_set.add(c)
        ss[vamp] = sire_set
    return ss

def pretty_print_sire_sets(ss, iw, vamps, newb):
```

.....

Pretty print sire sets for either unknown or newborn vampires.

vamps: subset of keys in ss to print.

newb: boolean, True for newborn language, False for unknown strata language.

If no vampires, print (None).

Otherwise:

For each vamp in sorted(vamps):

get infection window from iw: (start, end)

determine if single time unit or range

determine language:

- If newb: "X was sired by Y ..."

- Else: "X could have been sired by Y ..."

If multiple possible sires, use format_list_or.

.....

vamps = sorted(vamps)

if newb:

print("Newborn vampires:")

else:

print("Vampires of unknown strata:")

if len(vamps) == 0:

print(" (None)")

return

for vamp in vamps:

sires = sorted(ss[vamp])

start, end = iw[vamp]

if start == end:

timeframe = f"on day {str_time(start)}"

else:

timeframe = f"between day {str_time(start)} and day {str_time(end)}"

if len(sires) == 0:

if newb:

print(" (None)")

else:

print(" (None)")

else:

if len(sires) > 1:

s = format_list_or(sires)

```

else:
    s = sires[0]

if newb:
    print(f" {vamp} was sired by {s} {timeframe}.")
else:
    print(f" {vamp} could have been sired by {s} {timeframe}.")

```

Section 20: Hidden Vampire Detection

Function: find_hidden_vampires()

Here, the program uncovers vampires not explicitly revealed by the tests. If a newborn has exactly one possible sire, that sire must logically have been a vampire already. The function find_hidden_vampires() updates vks accordingly, ensuring no hidden contradictions remain in the infection model.

Code I wrote:

Section 20: vampire sire sets

```
def find_hidden_vampires(ss, iw, vamps, vks):
```

```
    """
```

If a newborn vampire has a single sire, that sire must be a vampire by the infection time. Additionally, propagate that sire's vampirism backward one unit to clarify the order.

Steps:

- For each vamp in vamps (newborn set):
 - If len(ss[vamp]) == 1:
 - Let sire = that one sire
 - Let (start,end) = iw[vamp] (start=end for newborn)
 - This means sire must be vampire by end time unit.
 - Mark sire as vampire at end and beyond, and also one unit earlier (AM period)
 - If conflicting with known human => error

Count how many changes made ($U \rightarrow V$).

Returns

```
    -----
```

```
    (vks, changes)
```

```
    """
```

```
    changes = 0
```

```
    participants = vks[0].keys()
```

```
    num_times = len(vks)
```

```
    for vamp in vamps:
```

```

if vamp in ss and len(ss[vamp]) == 1:
    sire = list(ss[vamp])[0]
    start, end = iw[vamp]

    for t in range(end, num_times):
        status = vks[t][sire]
        if status == "H":
            print("Error found in data: vampires cannot be humans; aborting.")
            sys.exit()
        elif status == "U":
            vks[t][sire] = "V"
            changes += 1

    if end > 0:
        t = end - 1
        status = vks[t][sire]
        if status == "H":
            print("Error found in data: vampires cannot be humans; aborting.")
            sys.exit()
        elif status == "U":
            vks[t][sire] = "V"
            changes += 1

return vks, changes

```

Section 21: Cyclic Analysis (Full System)

Function: `cyclic_analysis2()`

The final cyclic analysis combines all prior reasoning steps—recalculating infection windows (iw), sire data (ps), sire sets (ss), and strata (originals, vamps_unclear, newborns)—until a fully stable logical state is achieved. The result is a coherent, complete representation of the vampire infiltration pattern across all participants and days.

Code:

Section 21; done by professor

```

def cyclic_analysis2(vks, groups):
    count = 0
    changes = 1
    while (changes != 0):
        iw = find_infection_windows(vks)
        ps = find_potential_sires(iw, groups)
        vks, iw, ps, countz = cyclic_analysis(vks, iw, ps)
        o, u, n = vampire_strata(iw)

```

```
ss = calculate_sire_sets(ps)
vks, changes = find_hidden_vampires(ss, iw, n, vks)
count = count + 1
return (vks, iw, ps, ss, o, u, n, count)
```

Summary

The Vampire Hunting v1.4.4 program reconstructs the logical timeline of a simulated infection outbreak using daily test data and contact records. It systematically parses raw text into structured datasets, initializes temporal knowledge tables (vks), and applies forward and backward propagation to maintain internal consistency.

Through iterative reasoning, the program identifies infection windows (iw), determines potential sires (ps), refines them through cyclic analysis, and classifies participants into distinct strata—originals, newborns, and vamps_unclear. The final output presents a coherent, data-driven model that explains when infections occurred, who transmitted them, and how the contagion spread across time.